

I. Use Case Description	
Use Case Name	<i>Ontology-Guided Agentic Capabilities Orchestration</i>
Use Case Identifier	<i>AC-001</i>
Source	<i>CSCI 6340 - Ontologies</i>
Point of Contact	<i>Gia Oriana Santos (santog@rpi.edu), Barbara Kotlan (kotlab@rpi.edu), Jiaying(Michael) Wang (wangj63@rpi.edu), Jialin (Gary) Fang (fangj6@rpi.edu), Eli (guedee@rpi.edu)</i>
Creation / Revision Date	<i>9/30/2026</i>
Associated Documents	

II. Use Case Summary	
Goal	<i>To govern which agent skills and tools to invoke, as well as determine the most optimal orchestration plan an LLM should follow given the user's request and constraints</i>
Requirements	<i>This use case must accomplish the following tasks</i> ● <i>Identify agent capabilities that can fulfill the information requirements or goals expressed in a user request</i> ● <i>Construct one or more candidate orchestration plans by sequencing agent capabilities to satisfy the information requirements of a user request.</i> ● <i>Determine whether capabilities in a candidate plan have their required inputs, outputs, preconditions, and dependencies satisfied, and identify missing or incompatible dependencies before execution</i> ● <i>Select an orchestration plan that satisfies all or most of the constraints specified in the user's preferences</i> ● <i>Explain why an orchestration plan is selected by mapping candidate plans to decision criteria</i> ● <i>Plan independent capabilities in a given orchestration plan that can run in parallel</i>
Scope	● <i>These are the external servers available to the LLM</i> ○ <i>PubMed MCP</i> ○ <i>Supplement Advisor MCP</i> ○ <i>OpenFDA MCP</i> ○ <i>iHerb-CLI</i> ● <i>External servers are exclusively MCP and CLI</i> ● <i>The LLM cannot use RAG or GraphRAG</i> ● <i>Specific domain: personalized supplement recommendation</i> ● <i>Not considering "common knowledge" questions. Only questions related to personalized supplement recommendations</i>
Priority	<i>N/A</i>
Stakeholders	● <i>LLM users</i> ● <i>LLM agent developers</i> ● <i>External server providers</i>
Description	<i>The emergence of the MCP has enabled LLMs to access external tools and data sources through standardized interfaces, allowing AI agents to perform complex retrieval, reasoning, and decision-making tasks. As AI agents become increasingly modular, they must not only determine which external servers and tools to invoke, but also decide when specialized agentic skills should be executed to accomplish subtasks for an overall goal. As the number of available skills and tools continues to grow, LLM agents face increasingly difficult orchestration problems, including determining which skills and tools should be executed, dependencies between skills and tools in orchestration,</i>

II. Use Case Summary

choosing the best valid orchestration plan given the particular user and situation, and explaining why the orchestration plan should be preferred over another. Current LLMs typically make these decisions implicitly through prompting or learned behavior, making the orchestration process difficult to explain, reproduce, validate, and optimize.

This use case addresses the ambiguity of agentic skill and tool orchestration by incorporating an ontology as a layer between the user and agent to explicitly represent the capabilities, inputs, outputs, dependencies, execution requirements, and characteristics of available skills, external servers, and their associated tools. Given a user's request, the ontology guides the LLM agent in selecting the appropriate skills, identifying the relevant external servers and tools, determining the optimal sequence of skill executions and tool invocations, and selecting the orchestration plan that best satisfies the task requirements. The use case's scope focuses on generating evidence-based, personalized supplement recommendations by integrating information from a user's preference settings to act as constraints on orchestration plan selection. This use case also incorporates external servers to support retrieval from scientific literature, safety screening, and an existing product catalog. By grounding orchestration decisions in explicit ontological knowledge, the framework enables transparent, explainable, reusable, and repeatable agent workflows while ensuring that supplement recommendations are supported by user-specified constraints for semantic orchestration plan generation

Actors / Interfaces

Primary actors

- *LLM User*
- *LLM Agent*

Secondary actors (resources)

- *External Servers*
 - *PubMed MCP*
 - *Supplement Advisor MCP*
 - *OpenFDA MCP*
 - *iHerb-CLI*
- *NLP Tool*
- *Agent Harness (ie. Claude Code, Codex CLI, OpenClaw, etc.)*
- *Reasoner*
- *Knowledge Graph (KG)*

Pre-conditions

- *Ontology is loaded into the system*
- *LLM agent model must be able to use skills and tools (including MCP and CLI).*
- *LLM agent must have the following skills*
 - *Supplement Recommendation*
 - *Safety Screening*
 - *Evidence Verification*
 - *Product Selection*
- *LLM agent is connected to the following external servers*
 - *PubMed MCP*
 - *Supplement Advisor MCP*
 - *OpenFDA MCP*
 - *iHerb-CLI*
 - *LLM agents must also have access to the user's preference settings to facilitate constraint-aware selection*

Post-conditions

- *Harness provides ontology-aligned orchestration plan to the LLM*
- *Orchestration plan is recorded to the KG for provenance*

II. Use Case Summary

Triggers	• User sends a request to the LLM agent.
Performance Requirements	
Assumptions	
Open Issues	

III. Usage Scenarios

Provide at least two usage scenarios that flesh out the requirements outlined in the summary, including identification of requirements specific to any envisioned ontology or semantically-driven service or application. Scenarios should be described as narrative, with supporting diagrams as appropriate. In an Agile process, every user story relevant to the use case should be included and elaborated/rolled up into one or more usage scenarios, with a clear mapping from the user story to the scenario it is integrated in or mapped to.

1. Ken is a college student training to be a hybrid athlete in search of supplements while on a budget. Jessica is also looking for supplements, but is diagnosed with osteoarthritis and taking prior medication. Both use the Supplement Recommender agent through their LLMs, but they each have different user preferences in terms of agent orchestration.

Ken prefers to **minimize token usage and external tool calls** whenever possible. As a college student on a budget, he is looking for **affordable supplements** with **strong customer reviews** and **good value**. He wants recommendations that are **supported by scientific evidence but is willing to accept slightly less comprehensive analysis if it reduces cost**. Jessica, on the other hand, **prioritizes recommendations that are supported by high-quality scientific evidence**. She also expects the system to **screen for potential interactions with her prescription medication** before making any recommendations.

2. Dante uses the Supplement Recommender agent and can view the candidate orchestration plans considered by the agent through the system's harness. Although he can see the plans, he is unfamiliar with **what each capability does, which servers provide the invoked tools, what intermediate information artifacts are produced at each step of the orchestration, and why each agent capability was invoked in a particular order**. He wants to understand **why the agent selected one orchestration plan over the other candidate plans**.
3. Olivia wants to find the best heart-health supplement. She explicitly tells the Supplement Recommender agent to find **strong scientific evidence** while also requiring the response to use as **few external tool calls as possible**. The agent identifies that these preferences trade off against each other, because more evidence needs more tool calls, but they do not contradict each other. Regardless, the agent generates candidate plans with different **trade-offs between evidence coverage and execution cost**. The agent **determines which orchestration plan to choose based on Olivia's user preferences** and goes ahead to execute that orchestration plan.

IV. Basic Flow of Events

Narrative: *Often referred to as the primary scenario or course of events, the basic flow defines the process/data/work flow that would be followed if the use case were to follow its main plot from start to end. Error states or alternate states that might occur as a matter of course in fulfilling the use case should be included under Alternate Flow of Events, below. The basic flow should provide any reviewer a quick overview of how an implementation is intended to work. A summary paragraph should be included that provides such an overview (which can include lists, conversational analysis that captures stakeholder interview information, etc.), followed by more detail expressed via the table structure.*

In cases where the user scenarios are sufficiently different from one another, it may be helpful to describe the flow for each scenario independently, and then merge them together in a composite flow.

Usage Scenario 1 - Basic / Normal Flow of Events: constructing an orchestration plan for execution

Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request
2	-	NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints
4	-	Reasoner	Identifies candidate agent tools and skills from the information requirements
5	-	Reasoner	Determines candidate orchestration plans based on a combination of required agent tools and skills that satisfy the information requirements
6	-	Reasoner	Evaluates candidate orchestration plans by applying constraints (user constraints and execution constraints) and ranking them on the modeled decision criteria
7	-	Reasoner	Selects orchestration plan
8	-	Harness	Derives orchestration dependencies from the input/outputs of orchestration steps into a dependency graph
9	agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
10	agent		Generates answer to user's request
11		KG	Records skill and tool invocations from the orchestration as well as decision criteria as provenance

Summary: The **User** submits a request, after which the **NLP Tool** parses the request to extract goals, constraints, and preferences. The **Harness** retrieves the user's preference settings to identify orchestration constraints. The **Reasoner** identifies candidate tools and skills to satisfy the information requirements, as well as candidate orchestration plans that collectively satisfy those requirements. The **Reasoner** then evaluates and ranks the candidate plans according to user and planning constraints and relevant decision criteria before selecting an orchestration plan. The **Harness** derives dependencies between orchestration steps from their inputs and outputs to form a dependency graph, and the **Agent** executes the selected plan by invoking the required tools and skills in the appropriate order. The **Agent** then generates an answer to the user's request, while the **KG** records the tool and skill invocations and associated decision criteria as provenance for the resulting orchestration.

Usage Scenario 2 - Basic / Normal Flow of Events: explaining why an orchestration plan was selected for execution

Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request
2	-	Reasoner	Generates candidate orchestration plans
3	-	Reasoner	Selects orchestration plan based on DecisionCriteria
4		KG	Records candidate orchestration plans and selected plan with corresponding DecisionCriteria as provenance
5	Agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
6	user		Requests an explanation of the selected orchestration plan through the system's Harness
7		Reasoner	Identifies DecisionCriteria of corresponding selected

			orchestration plan
8	-	Reasoner	Maps each ToolInvocation and SkillInvocation to the Capability it provides and to the InformationRequirements and Constraints that Capability satisfies
9	-	Harness	Displays the DecisionCriteria and mapped InformationRequirements & Constraints to each agent capability in the orchestration plan
10	User	-	Views the explanation of the orchestration plan via the system's Harness

Summary: The **User** submits a request to the **Supplement Recommender Agent**. The **Reasoner** generates candidate orchestration plans and selects an orchestration plan based on applicable Decision Criteria, while the **Knowledge Graph** records the candidate plans, selected plan, and corresponding Decision Criteria as provenance. The **Agent** then executes the selected orchestration plan by invoking the required tools and skills according to the plan. Following execution, the **Reasoner** identifies the Decision Criteria associated with the selected plan and maps each Tool Invocation and Skill Invocation to the Capabilities they provide, as well as the Information Requirements and Constraints satisfied by those capabilities. The **Harness** then displays the Decision Criteria and the mapped Information Requirements and Constraints associated with each agent capability in the orchestration plan, allowing the **User** to view an explanation of why the selected orchestration plan was chosen.

Usage Scenario 3 - Basic / Normal Flow of Events: constraint-based selection based on non-conflicting Decision Criteria

Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request
2	-	NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints (ie. high evidence coverage, low external tool calls)
4	-	Reasoner	Identifies candidate orchestration plans whose Capabilities satisfy the InformationRequirements
5	-	Reasoner	Selects "Evidence Coverage" and "External Tool Calls" as DecisionCriteria to select orchestration plan
6	-	Harness	Calculates evidence coverage and external tool call scores for each candidate plan
7		KG	Records the evidence coverage and external tool call scores of each candidate plan
8	-	Reasoner	Ranks each candidate plan by evidence coverage and external tool call scores
9		Reasoner	Selects orchestration plan based on highest DecisionCriteria score
10	-	Harness	Derives orchestration dependencies from the input/outputs of orchestration steps into a dependency graph
11	agent	-	Executes orchestration plan by invoking agent tools and skills according to the plan
12	agent	-3	Generates answer to user's request
13		KG	Records candidate orchestration plans and selected plan with corresponding DecisionCriteria as provenance

Summary: The **User** submits a request to the **Supplement Recommender Agent**. The **NLP Tool** parses the request to extract the user's goals, constraints, and information requirements, while the **Harness** retrieves the user's preference settings to identify relevant orchestration constraints, such as *high evidence coverage* and *minimizing external tool calls*. The **Reasoner** identifies candidate orchestration plans whose Capabilities satisfy the extracted Information Requirements and selects *Evidence Coverage* and *External*

Tool Calls as the Decision Criteria for evaluating the plans. The **Harness** calculates the evidence coverage and external tool call scores for each candidate plan, and the **Knowledge Graph** records these scores as provenance. The **Reasoner** first drops any candidate plan that breaks a hard constraint, such as required safety screening. It then scores the remaining plans on each Decision Criterion and combines the scores using the priority the user gave each constraint; if the user gives no priorities, the criteria count equally, as in CQ 6. Olivia's evidence and tool-call constraints pull in different directions but can both be met to some degree, so the Reasoner trades them off in this weighted ranking instead of reporting a conflict. The **Harness** then derives the dependencies among the orchestration steps from their inputs and outputs to construct a dependency graph. The **Agent** executes the selected orchestration plan by invoking the required tools and skills according to the derived dependencies, generates an answer to the user's request, and the **Knowledge Graph** records the candidate plans, selected plan, Decision Criteria, and associated evaluation information as provenance.

V. Alternate Flow of Events

Narrative: *The alternate flow defines the process/data/work flow that would be followed if the use case enters an error or alternate state from the basic flow defined, above. A summary paragraph should be included that provides an overview of each alternate flow, followed by more detail expressed via the table structure.*

Usage Scenario 1 - Alternate Flow of Events: current agentic capabilities cannot satisfy the user's request			
Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request
2	-	NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints
4		Reasoner	Determines there are no agentic capabilities that satisfy one or more information requirements
5		Reasoner	Identifies unsatisfied information requirements
6	-	Harness	Returns unsatisfied information requirements
7	agent	-	Informs the users that the available agent capabilities cannot satisfy the user's request, highlighting the unsatisfied information requirements needed to answer the query

Summary: The **User** submits a request. The **NLP Tool** parses the request to extract goals, constraints, and information requirements. The **Harness** retrieves the user's preference settings to identify relevant orchestration constraints. The **Reasoner** determines that the available agentic capabilities cannot satisfy one or more of those requirements. It identifies the unsatisfied information requirements, which the **Harness** returns to the **Agent**. The **Agent** then informs the user that the currently available tools and skills cannot fully satisfy the request, explicitly highlighting the information requirements that cannot be fulfilled and are therefore needed to answer the query.

Usage Scenario 2 - Alternate Flow of Events: current agentic capabilities cannot satisfy the user's request (requires explanation)			
Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request
2	agent	-	Informs the users that the available agent capabilities cannot satisfy the user's request, highlighting the unsatisfied

			information requirements needed to answer the query
3	-	Reasoner	Retrieves the InformationRequirements the system was trying to satisfy in the previous query
4		Reasoner	Retrieve all the Tools and Skills available to the agent
5		Reasoner	Map those Tools and Skills to their Capabilities and the InformationRequirements they satisfy
6		Reasoner	Identifies unsatisfied InformationRequirements
7	-	Harness	Display the Tools and Skills with the InformationRequirements they satisfy, as well as the unsatisfied InformationRequirements
8	User	-	Views the explanation as to why the agent failed to execute the task via the system's Harness

Summary: When the available agentic capabilities cannot satisfy the **User's** request, the **Agent** informs the user that the request cannot be fulfilled and identifies the Information Requirements that remain unsatisfied. The **Reasoner** retrieves the Information Requirements associated with the request, as well as all Tools and Skills currently available to the agent. It then maps the available Tools and Skills to their corresponding Capabilities and determines which Information Requirements those Capabilities can satisfy, allowing the system to identify the Information Requirements that remain unsatisfied. The **Harness** displays the available Tools and Skills, the Information Requirements they can satisfy, and the unsatisfied Information Requirements. The **User** can then view this explanation through the system **Harness** to understand why the **Agent** was unable to execute the task.

Usage Scenario 3 - Alternate Flow of Events: identifies directly conflicting constraints (ex. Maximize tool use and minimize tool use)			
Step	Actor (Person)	Actor (System)	Description
1	user		Submits a request
2		NLP Tool	Parses request, extracting goals, constraints, and information requirements
3	-	Harness	Retrieves user's preference settings to identify orchestration constraints (ie. maximize tool use, minimize tool use)
4	-	Reasoner	Identifies that the constraints directly conflict with each other
5	-	Harness	Returns conflicting constraints
6	agent	-	Informs the user that it cannot execute the request because the constraints conflict, names the conflicting constraints, and asks the user to revise them

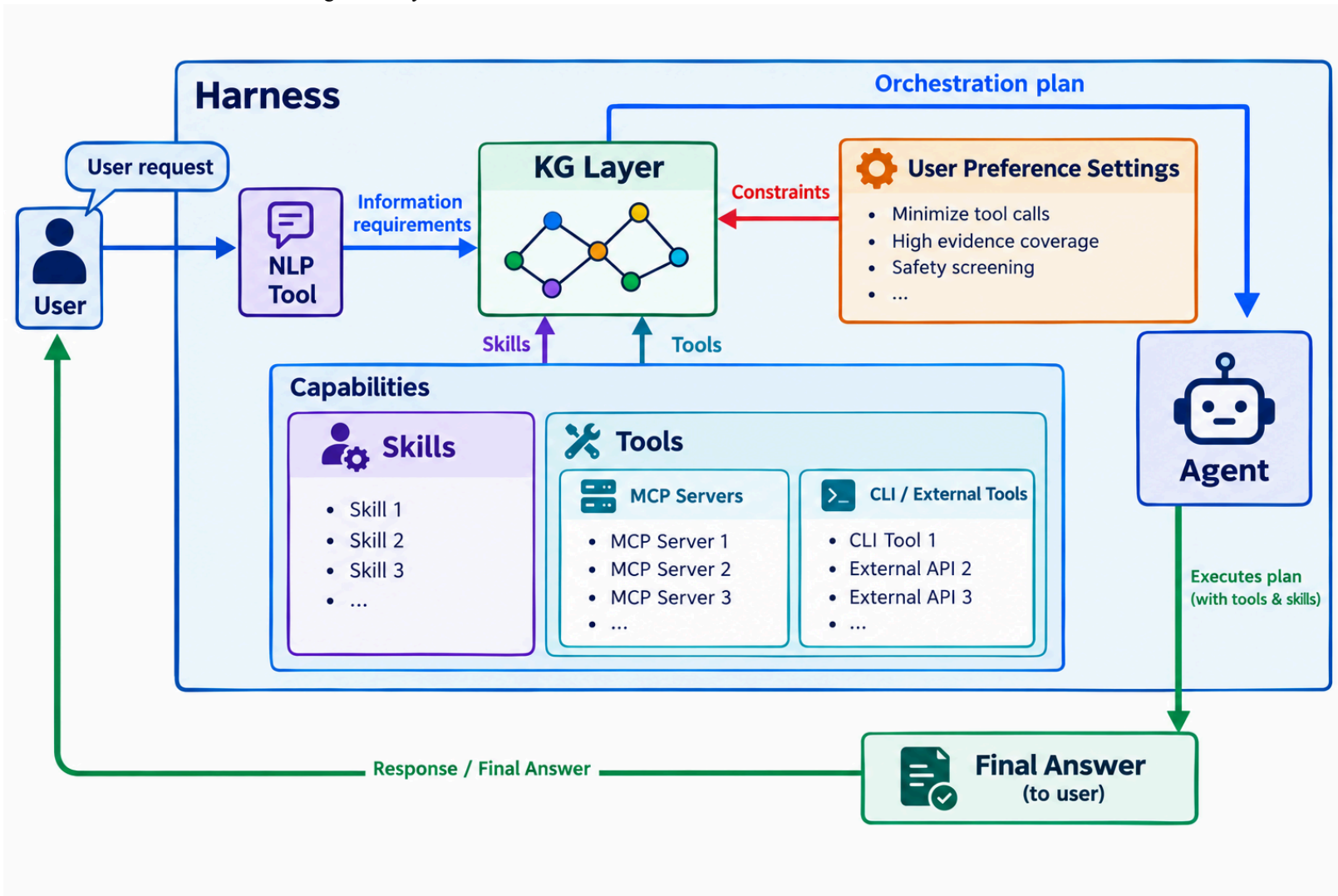
Summary: The **User** submits a request, and the **NLP Tool** parses the request to extract the user's goals, constraints, and Information Requirements. The **Harness** retrieves the user's preference settings to identify the orchestration constraints associated with the request, while the **Reasoner** evaluates these constraints and identifies a direct conflict

between them, such as *simultaneously maximizing and minimizing tool use*. The **Harness** returns the conflicting constraints, and the **Agent** informs the user that it cannot execute the task because the specified constraints are incompatible. The **User** is then prompted to resolve or modify the conflicting constraints before the request can be executed. Only constraints that cannot both hold, such as maximizing and minimizing tool use, stop planning; constraints that merely trade off are ranked as in the basic flow.

VI. Use Case and Activity Diagram(s)

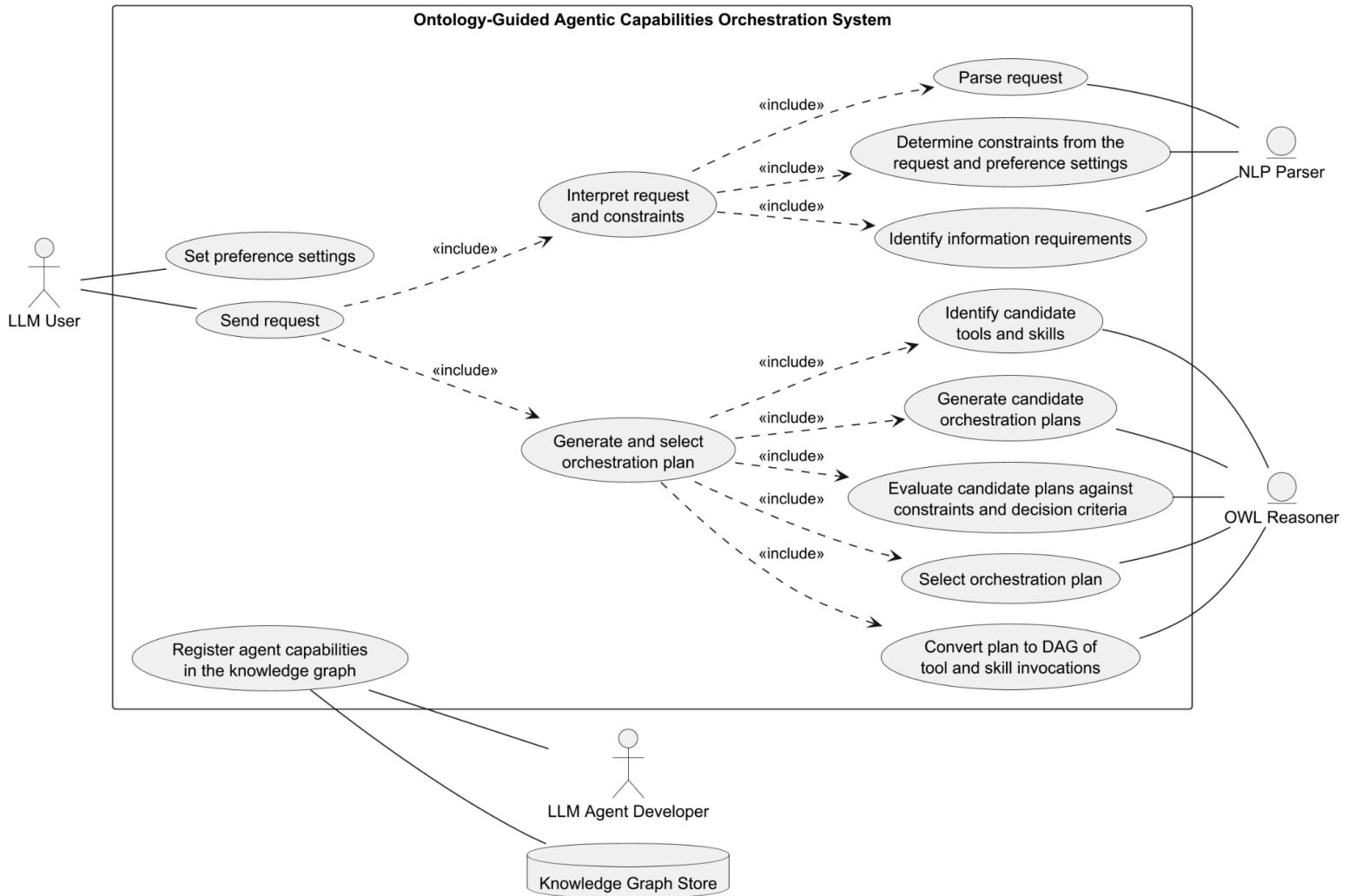
Provide the primary use case diagram, including actors, and a high-level activity diagram to show the flow of primary events that include/surround the use case. Subordinate diagrams that map the flow for each usage scenario should be included as appropriate

Structural Diagram of System Architecture



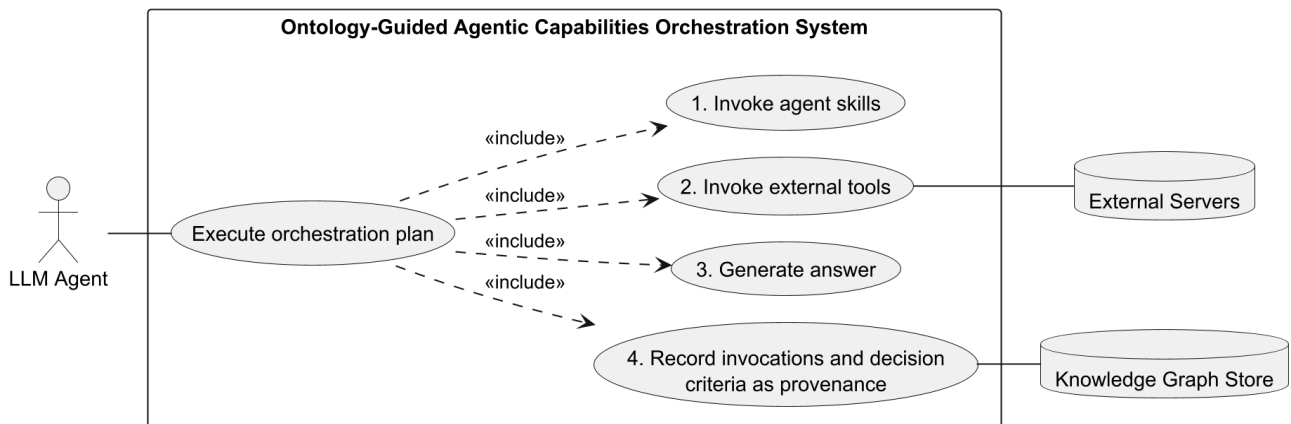
Use case diagram: LLM User and Agent Developer Side

Figure 2. Primary use case diagram - AC-001 Ontology-Guided Agentic Capabilities Orchestration: LLM User and Agent Developer Side



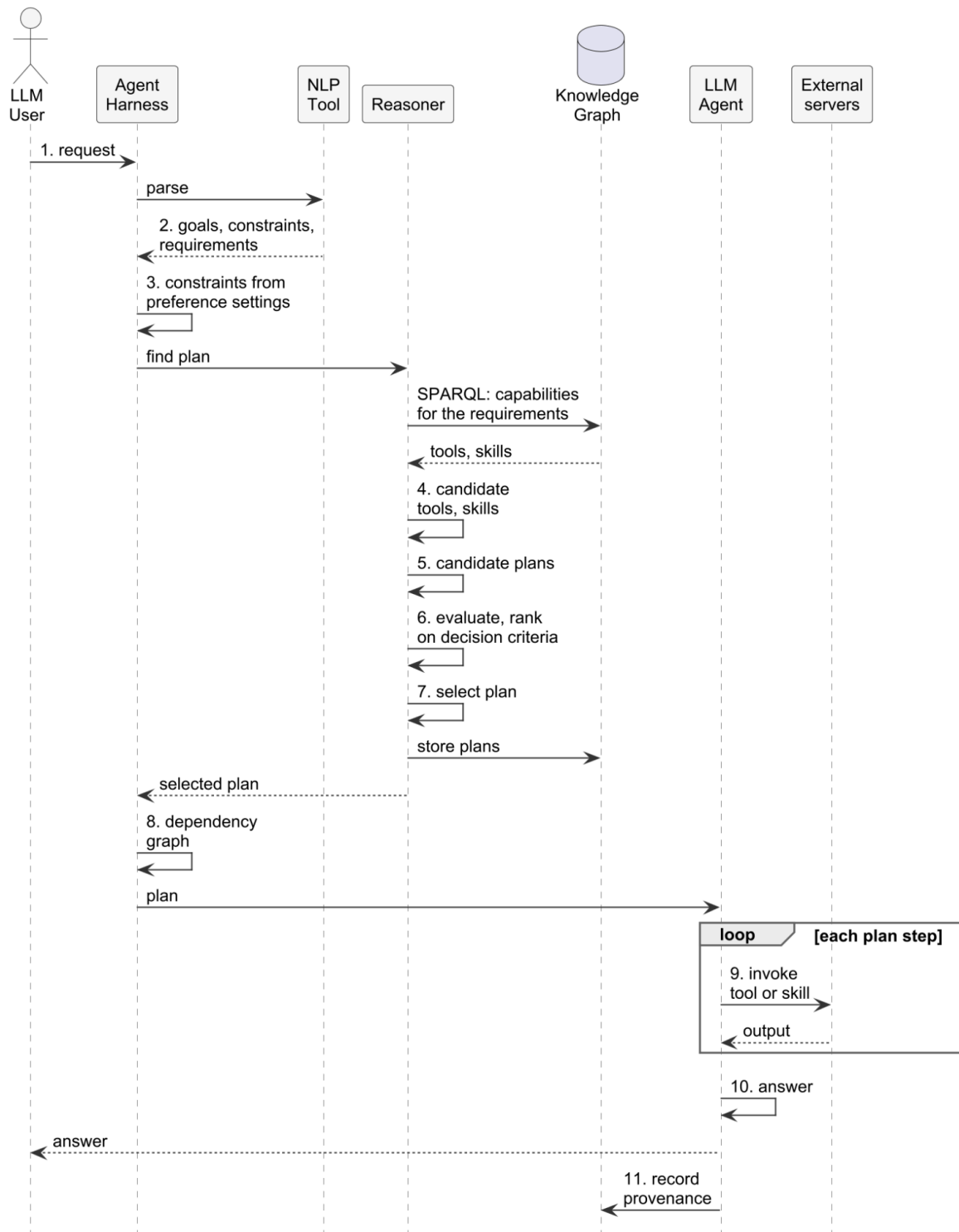
Use case diagram: Agent Side

Figure 2a. Primary use case diagram - AC-001 Ontology-Guided Agentic Capabilities Orchestration: Agent Side



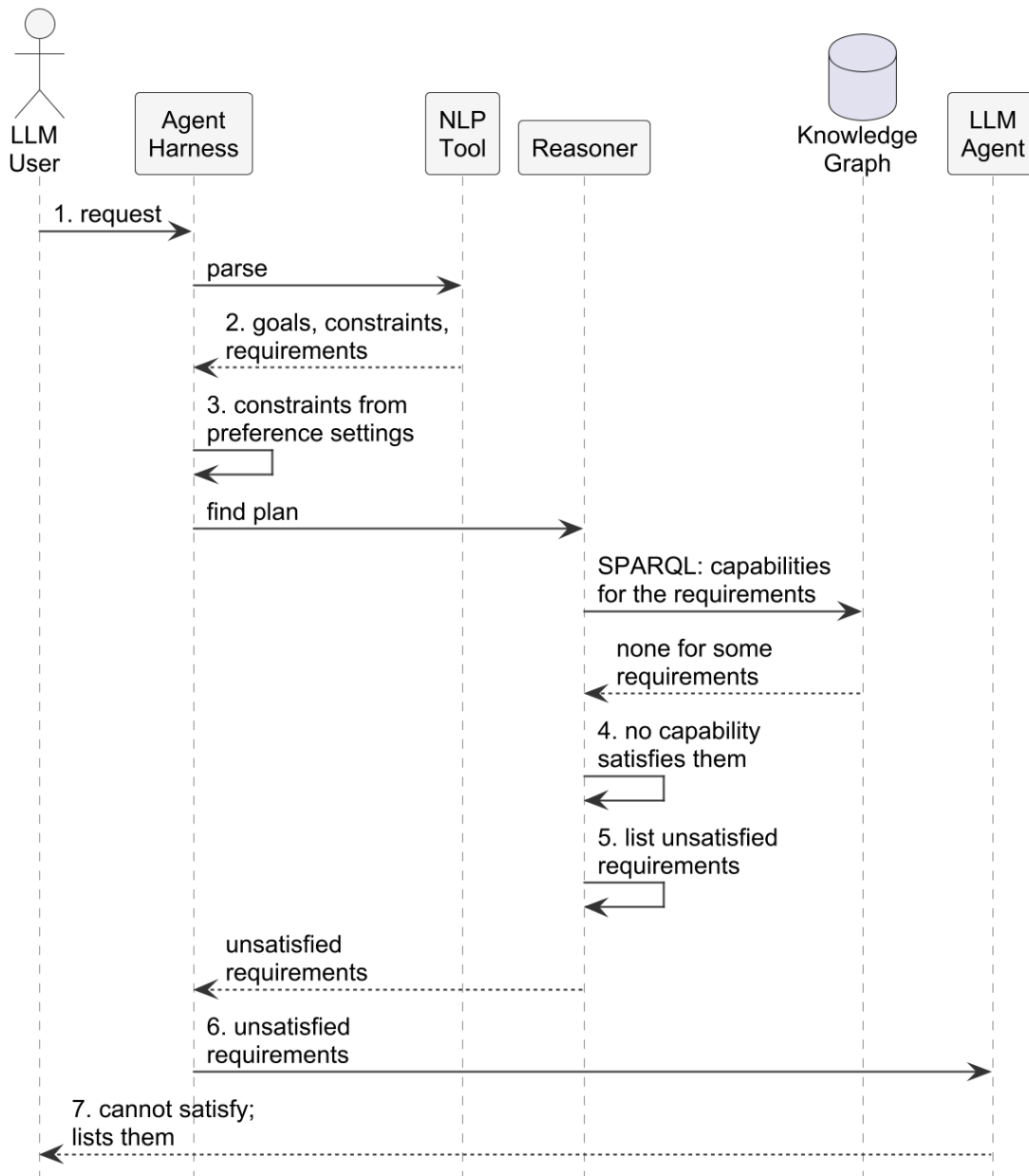
Basic flow Sequence Diagram

Basic Flow of Events (Usage Scenario 1): sequence diagram - AC-001



Alternate Flow Sequence Diagram: Current agentic capabilities cannot satisfy the user's request

Alternate Flow of Events (Usage Scenario 1): sequence diagram - AC-001



VII. Competency Questions

Provide at least 2 competency questions that you will ask of the vocabulary/ontology/knowledge base to implement this use case, including example answers to the questions.

Describe at least one way you expect to use the semantics and/or provenance to propose an answer to the questions. Include an initial description of why the semantics and/or provenance representation and reasoning provides an advantage over other obvious approaches to the problem. (optional – depending on the use case and need for supporting business case).

1. What orchestration plan should the agent follow to answer Ken's request for supplement recommendations, given his preference for few tool calls and low token use?

Refer to [usage scenario 1 \(Ken\)](#)

Query: provide me with supplement recommendations for building muscle mass

Candidate Answers:

1. **Get user profile**
2. **Get candidate supplements**
 - a. **Supplement_advisor: recommend_supplement(search_query) => supplement recommendations**
3. **Remove redundant supplements**
4. **Get supplement products**
 - a. **iHerb-CLI: search(supplement, sort=[price]) => list of supplement products**
5. **Rank supplement products by cost**
6. **Verify evidence**
 - a. **PubMed: search_articles(search_query) => PMIDs**
 - b. **PubMed: download_articles_batch(PMIDs) => article abstracts**
 - c. **summarize_evidence()**
7. **Rank supplement products by evidence**
8. **Select supplement product**
9. **Produce recommendations**

How to solve with ontology

1. System parses the user's request and retrieves any Constraints and InformationRequirements
 - a. InformationRequirements: supplement recommendations, supporting scientific evidence, product prices, customer reviews
2. System extracts the Constraints from the user's preference settings\
 - a. Constraints: minimize token usage, minimize external tool calls, include price-planning, include evidence verification, include customer reviews
3. System queries the KG to identify Tools and Skills that can satisfy each InformationRequirement
4. Ontology reasons over Tool and Skill dependencies, modeled by their expected inputs and outputs
 - a. Which tools and skills are needed?
 - b. What order they should execute in?
 - c. Which orchestration plan best satisfies the user's constraints?
5. System creates the Orchestration Plan as a DAG workflow graph of the Skills and Tools to execute in order
6. Orchestration Plan and related metadata are stored in KG for provenance
7. The system selects that orchestration plan for the agent to follow

2. Which workflow steps of a given orchestration should be executed in parallel while satisfying both workflow dependencies and the user's execution preferences?

Refer to [usage scenario 1 \(Jessica\)](#)

Suppose Jessica's orchestration plan is:

1. *Get user profile*
2. *Get candidate supplements*
 - a. *Supplement_advisor: recommend_supplement() => supplements*
3. *Screen interactions*
 - a. *OpenFDA: search_drug_adverse_events(medication/supplement)*
 - b. *OpenFDA: search_drug_labels(supplement)*
 - c. *OpenFDA: search_drug_recalls(supplement)*
4. *Rank by screening*
5. *Verify evidence*
 - a. *PubMed: search_articles(supplement/search_query) => PMIDs*

- b. *PubMed: download_articles_batch(PMIDs) => article abstracts*
 - c. *summarize_evidence()*
6. *Rank by evidence*
7. *Get supplement products*
 - a. *iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products*
8. *Rank supplement products by rating*
9. *Select supplement product*
10. *Produce recommendations*

Candidate Answer:

- **Screen interactions**
 - **OpenFDA: search_drug_adverse_events(medication/supplement)**
 - **OpenFDA: search_drug_labels(supplement)**
 - **OpenFDA: search_drug_recalls(supplement)**
- **Get supplement products**
 - **iHerb-CLI: search(supplement, sort=[rating]) => list of supplement products**
- **Verify evidence**
 - **PubMed: search_articles(supplement/search_query) => PMIDs**
 - **PubMed: download_articles_batch(PMIDs) => article abstracts**
 - **summarize_evidence()**

How to solve with ontology:

1. The system retrieves the user's **OrchestrationPlan** from the ontology-grounded knowledge graph.
2. The system constructs a workflow dependency graph by reasoning over the semantic relationships between workflow steps. Each **Skill** and **ToolInvocation** is linked to the **InformationArtifacts** it consumes and produces
 - a. Example InformationArtifact: supplement
3. The ontology infers dependency relationships between workflow steps. A workflow step is dependent on another if it consumes an **InformationArtifact** produced by that step.
4. The system identifies independent workflow branches by finding workflow steps whose required **InformationArtifacts** have already been satisfied and that do not consume outputs produced by one another.
5. The system returns the set of workflow steps eligible for concurrent execution, allowing the agent to execute those steps in parallel while preserving all semantic dependencies.

3. What skills are required to satisfy the user's request?

Refer to [*usage scenario 1 \(Jessica\)*](#)

Query: Provide me with supplement recommendations for building bone mass, while taking into account of my osteoarthritis and prescribed Celebrex medication

Candidate answer:

- **Personalized Supplement Recommendation Skill**
- **Supplement Safety Screening Skill (due to her current medication and diagnosed health condition)**
- **Evidence Verification Skill**
- **Product Selection Skill**

How to get the answer:

1. System processes the user query with the NLPTool to extract **InformationRequirements** to satisfy the user's request
2. System processes the user's preference settings to determine **Constraints** the user's **OrchestrationPlan** must abide by
3. System uses the KG to map what **Capabilities** satisfy the **InformationRequirements** and the **Skills** that perform those **Capabilities**.

4. System returns the **Skills** required to satisfy the user's request

4. Why were the agent capabilities for a given orchestration invoked in this order?

Refer to [usage scenario 2](#)

Query: "Compare magnesium glycinate and magnesium citrate for supporting sleep, and summarize the scientific evidence for each."

Dante's constraints: minimize execution time, safety screening

Suppose the chosen orchestration plan is:

1a. Verify evidence

PubMed: search_articles(magnesium glycinate for sleep) => PMIDs

PubMed: download_articles_batch(PMIDs) => article abstracts

summarize_evidence()

1b. Screen interactions

OpenFDA: search_drug_adverse_events(magnesium glycinate)

OpenFDA: search_drug_labels(magnesium glycinate)

OpenFDA: search_drug_recalls(magnesium glycinate)

2a. Verify evidence

PubMed: search_articles(magnesium citrate for sleep) => PMIDs

PubMed: download_articles_batch(PMIDs) => article abstracts

summarize_evidence()

2b. Screen interactions

OpenFDA: search_drug_adverse_events(magnesium citrate)

OpenFDA: search_drug_labels(magnesium citrate)

OpenFDA: search_drug_recalls(magnesium citrate)

3. Synthesize evidence

Candidate Answer:

One of Dante's constraints is safety screening. Because the Screen interactions skill satisfies that constraint, it must be included in the orchestration plan. Another one of Dante's constraints is to minimize execution time. Since the Verify evidence and Screen interactions skills require the same dependencies (candidate supplement), they can be invoked in parallel to minimize execution cost. Doing so satisfies both constraints specified by Dante.

Why it's hard without an ontology:

- An execution log or a relational table records that Verify evidence and Screen interactions ran at the same time, but not why: it holds no statement that both consume the same CandidateSupplement artifact and that neither consumes the other's output, nor that "minimize execution time" and "safety screening" are the constraints that made this ordering preferable.
- Answering "why this order" requires the dependency relations between steps and the mapping from constraints to decision criteria to exist as explicit, queryable facts. The ontology supplies both.

How to get the answer:

1. The system retrieves Dante's OrchestrationPlan from the KG together with the InformationRequirements and Constraints extracted from his request and execution settings.
2. For every SkillInvocation and ToolInvocation in the plan, the system reads the InformationArtifacts it consumes and produces: Verify evidence consumes CandidateSupplement and produces EvidenceAssessment; Screen interactions consumes CandidateSupplement and produces SafetyAssessment; Synthesize evidence consumes both assessments.
3. The ontology infers the dependency relation: a step depends on another step only if it consumes an InformationArtifact that the other produces. Verify evidence and Screen interactions consume nothing from each other, so the reasoner classifies them as independent; Synthesize evidence depends on both, so it must run last.

4. The system traces each Constraint to the DecisionCriterion it selects. "Safety screening" is satisfied only by the Screen interactions Capability, which is why that skill is in the plan; "minimize execution time" selects the execution-time criterion, which ranks a plan that runs the two independent steps in parallel above the sequential alternative.
5. The system returns the ordering explanation: which steps were independent (from the inferred dependencies) and which Constraints and DecisionCriteria justified the parallel branch, all read from the provenance the KG recorded when the plan was selected (basic flow step 11).

5. Why was each agent capability chosen to execute in a given orchestration plan?

Refer to [usage scenario 2](#)

Use query, constraints, and orchestration plan from this [example](#)

Candidate answer:

- **Verify evidence skill and PubMed: satisfied InformationRequirement "summarize scientific evidence"**
- **Screen interactions skill and OpenFDA: satisfied constraint "safety screening"**
- **Synthesize evidence skill: satisfied InformationRequirement "compare magnesium glycinate and magnesium citrate"**

How to get the answer:

1. The system retrieves Dante's selected plan and, for each step, the task, the Capability the task requires, and the skill or tool that performed it.
2. For each Capability, it follows "satisfies" to the InformationRequirements and Constraints of Dante's request:
 - a. The evidence-verification capability satisfies "summarize scientific evidence".
 - b. The interaction-screening capability satisfies the constraint "safety screening".
 - c. The evidence-synthesis capability satisfies "compare magnesium glycinate and magnesium citrate".
3. A step whose capability satisfies nothing in the request is unnecessary. Every step in Dante's plan traces to at least one requirement or constraint.
4. The answer lists, for each step, the capability and what it satisfied, read from the KG record of the selected plan.

6. Why was orchestration plan B selected over orchestration plan A?

Refer to [usage scenario 3](#)

Plan A:

Get candidate supplements

- a. *Supplement advisor: recommend_supplement(heart health) => supplement recommendations*

Verify evidence

- b. *PubMed: search_articles(search_query) => PMIDs*
- c. *PubMed: download_articles_batch(PMIDs) => article abstracts*
- d. *summarize_evidence()*

Screen interactions

- e. *OpenFDA: search_drug_adverse_events(medication/supplement)*
- f. *OpenFDA: search_drug_labels(supplement)*
- g. *OpenFDA: search_drug_recalls(supplement)*

Verify evidence (second, duplicate pass)

- h. *PubMed: search_articles(search_query) => PMIDs*
- i. *PubMed: download_articles_batch(PMIDs) => article abstracts*
- j. *summarize_evidence()*

Rank by evidence

Select supplement product

Produce recommendations

Plan B:

Get candidate supplements

- k. *Supplement_advisor: recommend_supplement(heart health) => supplement recommendations*

Verify evidence

- l. *PubMed: search_articles(search_query) => PMIDs*
- m. *PubMed: download_articles_batch(PMIDs) => article abstracts*
- n. *summarize_evidence()*

Rank by evidence

Select supplement product

Produce recommendations

Candidate Answer:

Both plans call the Supplement Advisor MCP and verify the candidate supplements against PubMed, so both satisfy Olivia's "strong scientific evidence" constraint. Plan A additionally screens interactions with OpenFDA, which Olivia did not ask for, and runs a second Verify evidence pass with a broader search query, which finds supporting studies for every candidate supplement (evidence coverage 1.00, against 0.70 for Plan B). These extra steps cost five more external tool calls: Plan A makes 8 where Plan B makes 3. Plan B satisfies the evidence constraint with the fewest tool calls, ranks first on the combined DecisionCriteria, and is selected. Plan A is kept in the KG as a rejected candidate together with its scores (see CQ 9).

Why it's hard without an ontology:

- Multiple criteria in choosing the most optimal orchestration plan (information requirements, tool capabilities, domain authority, dependency constraints, execution cost, and user constraints)
- Can traverse to find multiple candidate orchestration plans that satisfy the user's request
 - difficult to systematically compare and rank them without explicit semantic representations
- Explanations require traceable relationships between requirements, tool capabilities, planning objectives, and constraints.

How to get the answer:

1. The system traverses the KG of all the candidate orchestration plans for a given query, assuming that the candidate plans already satisfy the **InformationRequirements** for that query
2. The system uses the ontology to evaluate them according to the **DecisionCriteria** specified by the user's **Constraints**
 - a. **DecisionCriteria**: high evidence coverage, low tool usage
3. The system ranks each candidate plan by the chosen **DecisionCriteria**
 - a. Plan A:
 - i. High Evidence Coverage: 1.00
 - ii. Low Tool Usage: 0.00
 - b. Plan B:
 - i. High Evidence Coverage: 0.70
 - ii. Low Tool Usage: 1.00
 - c. Both scores run from 0 to 1 across the candidate plans. Low Tool Usage = (most calls – this plan's calls) / (most calls – fewest calls), so Plan A (8 calls) scores 0.00 and Plan B (3 calls) scores 1.00. Olivia gave both constraints the same priority, so the two scores are added with equal weights.
4. The system chooses the highest-ranked plan based on the cumulative **DecisionCriteria** scores.
 - a. Plan A: $1.00 + 0.00 = 1.00$
5. Plan B: $0.70 + 1.00 = 1.70$
6. The system records the chosen orchestration plan, the DecisionCriteria supporting its selection, and all alternative orchestration plans as instances in the KG for provenance-awareness

7. Which constraints were violated by a candidate orchestration plan?

Refer to [usage scenario 3](#), and the candidate plan A from this [example](#)

Candidate Answer:

The candidate plan violated the “minimize external tool calls” constraint. While it did satisfy the “evidence coverage” constraint, it uses some unnecessary tool calls (the three OpenFDA calls screen interactions, which Olivia did not ask for, and the second PubMed pass adds evidence at the cost of two more calls) making it rank lower than other candidate plans.

How to get the answer:

1. The system retrieves Olivia's candidate plans and their evaluations: one score per plan and decision criterion.
2. Hard constraints come first. Plan A breaks none, so it was not excluded.
3. For each criterion, it compares Plan A's score with the selected Plan B's. Plan A scores 0.00 on Low Tool Usage against B's 1.00.
4. **It follows that criterion back to the constraint it measures: "minimize external tool calls".** That is the constraint Plan A violated. Its higher evidence score did not make up for it, because both constraints carry equal priority.

8. What skills depend on other skills? (for testing ontology correctness)

The Evidence Verification Skill, Supplement Safety Screening Skill, and Product Selection Skill all depend on the Personalized Supplement Recommendation Skill in order to execute. All of those skills require a CandidateSupplement as input, which is only returned by the Personalized Supplement Recommendation Skill as output.

How to get the answer:

1. System traverses the ontology-grounded KG and identifies all available Skills
2. The system retrieved produced and consumed Information Artifacts for each Skill. Ex.
 - a. Personalized Supplement Recommendation Skill → produces CandidateSupplement
 - b. Evidence Verification Skill → consumes CandidateSupplement, produces EvidenceAssessment
 - c. Safety Screening Skill → consumes CandidateSupplement, produces SafetyAssessment
 - d. Product Selection Skill → consumes EvidenceAssessment, SafetyAssessment, and ProductList
3. Ontology reasons over semantic relationships between Skills and InformationArtifacts to infer skill dependencies. Skill B is dependent on Skill A if B consumes an Information Artifact produced by A.
4. The system constructs a Skill dependency graph of inferred relationships
5. The system returns the set of Skills that must be executed sequentially from the dependency graph.

9. Are all the prerequisites satisfied before executing this Skill given the agent's current phase in orchestration? (for testing ontology correctness)

Example: about to execute Evidence Verification Skill when:

- HealthProfile ✓
- CandidateSupplement ✗
- SearchQuery ✓

No, the agent cannot execute the Evidence Verification Skill because not all the prerequisites have been satisfied (namely CandidateSupplement). The agent must call the Personalized Supplement Recommendation Skill, which returns a CandidateSupplement before executing the Evidence Verification Skill.

How to get the answer:

1. System retrieves the agent's current phase in the orchestration plan and identifies the Skill it's trying to execute.

2. System retrieves the Skill's prerequisites by querying the Skill dependency graph mentioned in CQ 7.
3. System checks off which prerequisites have been satisfied based on the agent's current orchestration state. Prerequisites are InformationArtifacts that are required as input for the Skill
4. If any unsatisfied prerequisite InformationArtifacts are detected, the system traces those artifacts to their producing Skills
5. The system returns one of the following two states:
 - a. Executable: all prerequisites are satisfied
 - b. Blocked: one or more prerequisites are missing but can be satisfied by calling another Skill

VIII. Resources

In order to support the capabilities described in this Use Case, a set of resources must be available and/or configured. These resources include the set of actors listed above, with additional detail, and any other ancillary systems, sensors, or services that are relevant to the problem/use case.

Knowledge Bases, Repositories, or other Data Sources

Data	Type	Characteristics	Description	Owner	Source	Access Policies & Usage
<i>(dataset or repository name)</i>	<i>(remote, local/in situ, etc.)</i>	<i>e.g. – no cloud cover</i>	<i>Short description of the dataset, possibly including rationale of the usage characteristics</i>		<i>Source (possibly a system, or remote site) for discovery and access</i>	
PubMed MCP Server	<i>remote</i>	<i>Third-party MCP server</i>	<i>Connects to research repository for retrieving biomedical-related papers</i>	<i>aeghn nsw</i>	<i>https://github.com/aeghnnsnsw/pubmed-mcp</i>	<i>Open-source Github License - must comply with PubMed's terms of service for API usage guidelines</i>
Supplement Advisor MCP Server	<i>remote</i>	<i>Third-party MCP server</i>	<i>Recommends supplements based on user's needs</i>	<i>erinheit451</i>	<i>https://github.com/erinheit451/supplement-advisor-mcp</i>	<i>Open-source Github MIT license</i>
OpenFDA MCP Server	<i>remote</i>	<i>Third-party MCP server</i>	<i>Provides safety and regulatory information on FDA-regulated products</i>	<i>Augmented Nature</i>	<i>https://github.com/Augmented-Nature/OpenFDA-MCP-Server</i>	<i>Open-source Github license</i>

			(recalls, adverse effects, shortages)			
<i>iHerb-CLI</i>	<i>remote</i>	<i>Third-party Client</i>	<i>Provides pricing and reviews for supplements from iHerb</i>	<u>SeverinAlexB</u>	<u>https://github.com/SeverinAlexB/iherb-cli</u>	<i>Open-source github MIT license</i>

External Ontologies, Vocabularies, or other Model Services

Resource	Language	Description	Owner	Source	Describes/Uses	Access Policies & Usage
<i>(ontology, vocabulary, or model name)</i>	<i>(ontology language and syntactic form, e.g., RDFS - N3)</i>	<i>If the service is one that runs a given ontology or model-based application at a given frequency, state that in addition to the basic description</i>		<i>Source (link to the registry or directly to the ontology, vocabulary, or model where that model is maintained, if available)</i>	<i>List of one or more data sources described by and/or used by the model</i>	
Representing Agentic Tools in Knowledge Graphs for Structure-Aware Tool Discovery Under Tool Overload	Paper that proposes high-level ontology (core classes, relationships, taxonomies) No access to github or owl/rdf file	Proposes ontology and KG as layer for tool discovery	Isaiah Onando Mulang' ~Isaiah_Onando_Mulang'l , Johannes Thaller, Tushar Trivedi, Lars Heling, Felix Sasaki	<u>https://openreview.net/forum?id=7MVoH9Y3mi&noteId=LGfMuSRiu6</u>	Use cases: Discovering tools	Creative Commons License Attribution 4.0 International (CC BY 4.0).
AgentO	Ontology file (RDF/OWL)	Formalizes core concepts, roles, and relationships necessary to describe contemporary agent-based and workflow-oriented AI systems, with particular	Kabul Kurniawan, Fajar J. Ekaputra	<u>https://sepses.ifs.tuwien.ac.at/ontology/index-en.html</u>	Use cases: Workflow reconstruction Workflow reuse Workflow auditing Captured classes: Agent, Task, Tool, Goal, Role, Capability,	Creative Commons License Attribution 4.0 International (CC BY 4.0).

		emphasis on LLM-driven agent frameworks			Workflow Pattern, Workflow Step	
--	--	---	--	--	---------------------------------	--

Other Resources, Service, or Triggers (e.g., event notification services, application services, etc.)

Resource	Type	Description	Owner	Source	Access Policies & Usage
<i>(sensor or external service name)</i>		<i>Include a description of the resource as well as availability, if applicable</i>	<i>Primary owner of the service</i>	<i>Application or service URL; if subscription based, include subscription and any subscription owner</i>	
Open WebUI	<i>Agent Harness</i>	<i>Web interface to connect AI models Supports MCP and Agent Skills</i>	<i>Tim J. Baek and team</i>	<i>https://knowledge.exlibrisgroup.com/Ulrich's/Product_Documentation/Configuring/Ulrichsweb_API/Ulrichsweb%3A_Using_the_Ulrichsweb_API</i>	<i>Open WebUI License</i>

IX. References and Bibliography

List all reference documents – policy documents, regulations, standards, de-facto standards, glossaries, dictionaries and thesauri, taxonomies, and any other reference materials considered relevant to the use case

- [1] I. O. Mulang', J. Thaller, T. Trivedi, L. Heling, and F. Sasaki, "Representing agentic tools in knowledge graphs for structure-aware tool discovery under tool overload," in Proc. 2nd Workshop on Knowledge Graphs and Agentic Systems Interplay (NORA 2026), co-located with IJCAI-ECAI 2026, Bremen, Germany, 2026. [Online]. Available: <https://openreview.net/forum?id=7MVoH9Y3mi>
- [2] A. Ekelhart, K. Kurniawan, F. J. Ekaputra, and E. Kiesling, "AgentO: An ontology for modeling agentic AI systems," in The Semantic Web: 23rd European Semantic Web Conference, ESWC 2026, Proceedings, Part II, Lecture Notes in Computer Science, vol. 16550, M. Acosta et al., Eds. Cham, Switzerland: Springer, 2026, pp. 298–320, doi: 10.1007/978-3-032-25159-6_16.
- [3] K. Kurniawan and F. J. Ekaputra, "AgentO: An ontology for modeling agentic AI systems," ontology release v0.2, Zenodo, Jan. 22, 2026, doi: 10.5281/zenodo.18342625. [Online]. Available: <https://w3id.org/agentic-ai/onto>
- [4] Model Context Protocol Project, "Model Context Protocol specification," revision 2026-07-28, LF Projects, LLC, Jul. 28, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2026-07-28>
- [5] X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, et al., "SkillsBench: Benchmarking how well agent skills

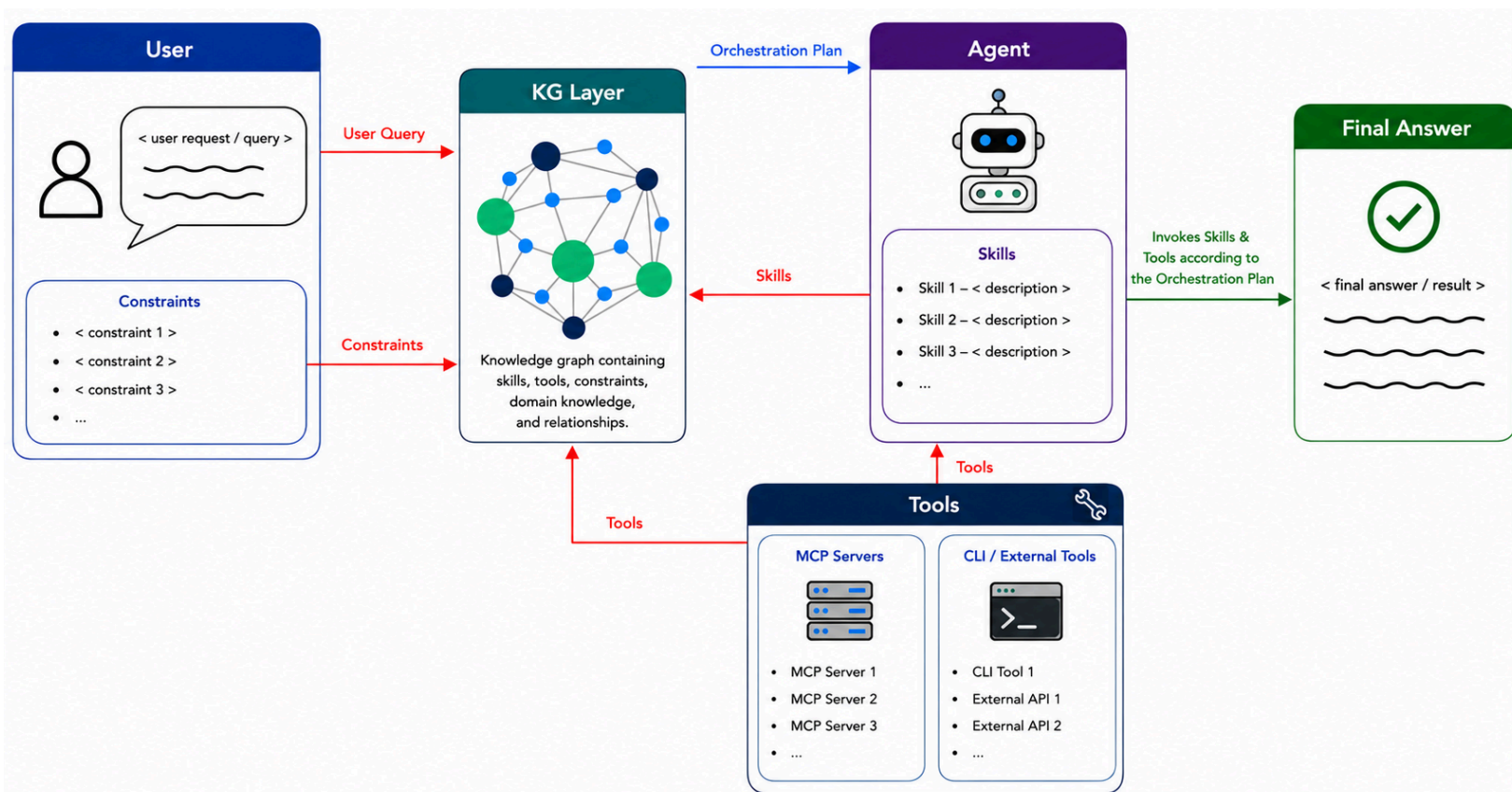
- work across diverse tasks,” arXiv:2602.12670, Feb. 2026. [Online]. Available: <https://arxiv.org/abs/2602.12670>
- [6] H. Xu, C. Li, X. Ma, X. Ou, Z. Zhang, T. He, et al., “The evolution of tool use in LLM agents: From single-tool call to multi-tool orchestration,” arXiv:2603.22862, Mar. 2026. [Online]. Available: <https://arxiv.org/abs/2603.22862>
- [7] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, “Understanding the planning of LLM agents: A survey,” arXiv:2402.02716, Feb. 2024. [Online]. Available: <https://arxiv.org/abs/2402.02716>
- [8] D. Martin et al., “OWL-S: Semantic markup for web services,” W3C Member Submission, Nov. 22, 2004. [Online]. Available: <https://www.w3.org/submissions/OWL-S/>
- [9] T. Lebo, S. Sahoo, and D. McGuinness, Eds., “PROV-O: The PROV ontology,” W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/prov-o/>
- [10] National Institute of Standards and Technology, “Lessons learned from the consortium: Tool use in agent systems,” Aug. 5, 2025. [Online]. Available: <https://www.nist.gov/news-events/news/2025/08/lessons-learned-consortium-tool-use-agent-systems>
- [11] IBM Research, “AI Atlas Nexus: AI risk ontology.” [Online]. Available: <https://ibm.github.io/ai-atlas-nexus/ontology/>
- [12] D. Garijo and Y. Gil, “The P-Plan ontology,” version 1.3, Mar. 12, 2014. [Online]. Available: <http://purl.org/net/p-plan>
- [13] N. Noy and A. Rector, Eds., “Defining N-ary relations on the Semantic Web,” W3C Working Group Note, Apr. 12, 2006. [Online]. Available: <https://www.w3.org/TR/swbp-n-aryRelations/>
- [14] J. Bao, E. F. Kendall, D. L. McGuinness, and P. F. Patel-Schneider, Eds., “OWL 2 Web Ontology Language quick reference guide (second edition),” W3C Recommendation, Dec. 11, 2012. [Online]. Available: <https://www.w3.org/TR/owl2-quick-reference/>

X. Notes

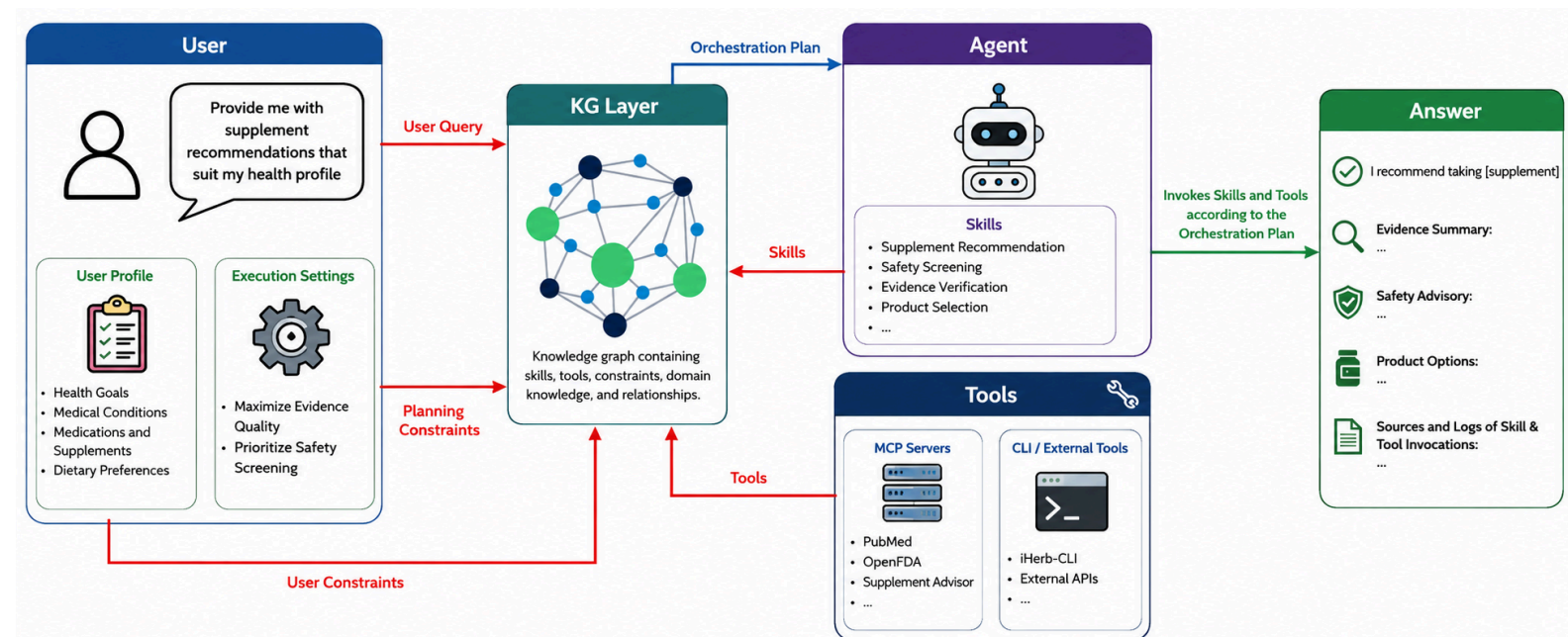
There is always some piece of information that is required that has no other place to go. This is the place for that information.

Concept of Operations

For a general ontology-guided agentic orchestration use case



For a specific scope: personalized supplement recommendation



Future work

- Ontology will consider user-aware personalized constraints